

Atividade Prática 01: Análise e Refatoração de Sistemas Legados

Disciplina: Design de Software

Contexto: Manutenção de Software e Qualidade de Código

1. Cenário de Negócio

Você assumiu a responsabilidade por um módulo crítico de um sistema de vendas. A equipe anterior implementou uma funcionalidade de "Finalização de Pedido" que, embora funcione, tem gerado altos custos de manutenção.

A diretoria solicitou duas alterações urgentes:

1. A introdução de uma nova regra de cálculo de frete baseada em regiões, e não apenas no tipo de envio.
2. A migração do mecanismo de log, que deixará de ser em console para ser em arquivo.

Ao abrir a classe responsável, você percebe que qualquer alteração corre o risco de quebrar funcionalidades não relacionadas.

2. Fundamentação Teórica

Antes de iniciar, recorde os dois conceitos que guiarão sua refatoração:

- **Coesão (O Princípio da Responsabilidade Única):** Uma classe deve ter apenas uma razão para mudar. Se você precisa abrir a mesma classe para ajustar uma regra tributária, corrigir um erro de conexão com o banco e alterar o texto de um e-mail, essa classe tem **baixa coesão**. Ela sabe demais e faz demais.
- **Acoplamento (O Princípio da Independência):** Refere-se ao grau de dependência direta entre módulos. Se a sua classe de negócio instancia diretamente (new) uma classe de banco de dados, ela está **fortemente acoplada**. O ideal é que a classe de negócio dependa de abstrações ou que receba as dependências prontas, sem saber como foram criadas.

3. O Desafio (Código Legado)

Analise a classe OrderManager abaixo. Ela viola flagrantemente os princípios de Design de Software. Não há comentários indicando onde estão os erros; você deve ler o código e identificar onde as responsabilidades se misturam.

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;
```

```

public class OrderManager {

    public void processOrder(int orderId, double amount, String region, String customerEmail) {

        // Bloco A
        double shippingCost = 0;
        if (region.equalsIgnoreCase("SUL")) {
            shippingCost = 20.00;
        } else if (region.equalsIgnoreCase("NORTE")) {
            shippingCost = 45.00;
        } else {
            shippingCost = 30.00;
        }

        if (amount > 500) {
            shippingCost = 0;
        }

        double finalAmount = amount + shippingCost;

        // Bloco B
        try {
            Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/shop",
"user", "pass");
            String sql = "INSERT INTO orders (id, total, region) VALUES (?, ?, ?)";
            PreparedStatement stmt = conn.prepareStatement(sql);
            stmt.setInt(1, orderId);
            stmt.setDouble(2, finalAmount);
            stmt.setString(3, region);
            stmt.execute();
            conn.close();
        } catch (Exception e) {
            e.printStackTrace();
        }

        // Bloco C
        System.out.println("--- EMAIL DISPATCHER ---");
        System.out.println("To: " + customerEmail);
        System.out.println("Subject: Order Confirmed");
        System.out.println("Body: Your order #" + orderId + " is confirmed. Total: " + finalAmount);
        System.out.println("-----");
    }
}

```

4. Roteiro da Atividade

Parte A: Diagnóstico

Identifique e anote:

1. Quais são as três responsabilidades distintas que a classe OrderManager assumiu para si indevidamente?
2. O que acontece com a lógica de negócio (cálculo de frete) se o banco de dados cair ou a senha do banco mudar? Isso é aceitável?

Parte B: Refatoração

Reescreva a solução em Java segregando as responsabilidades.

- Não utilize frameworks (como Spring ou Hibernate) agora; faça a injeção de dependência manualmente via construtor.
- Crie classes especialistas para cada responsabilidade identificada.
- A classe OrderManager (ou OrderService) deve apenas **coordenar** o fluxo.